



# **TED UNIVERSITY**

## **CMPE 492 – Senior Project II Final Report**

### **Team Members**

Tolga AKYOL

Yiğit AYDIN

Yusuf KAYNAKÇI

### **Supervisor**

Dr. Evren Coşkun

### **Jury Members**

Prof. Dr. Tansel Dökeroğlu, Dr. Deniz Cantürk

Date: 15/05/2026

# SmartLeaf

## 1. Abstract

SmartLeaf is an artificial intelligence based plant health analysis system developed as a senior design project. The main purpose of the system is to support farmers, agricultural workers, researchers, and non-technical users by providing an accessible web-based tool for preliminary plant health assessment through leaf images. The system allows users to upload a plant leaf image and receive automated outputs such as disease classification, confidence score, health score, and possible nutrient deficiency indication.

The project combines web technologies, backend API development, image processing, and machine learning based inference in a modular architecture. The final system includes a frontend deployed on Vercel, a backend service deployed on Render, and an analysis workflow that validates uploaded images, processes them through the machine learning service, and presents the results through a user-friendly interface. SmartLeaf also provides structured result presentation and report generation functionality to make the analysis output easier to understand and share.

During the project, the team focused not only on technical implementation but also on the social, economic, environmental, and ethical impact of the system. SmartLeaf is designed as a decision-support tool rather than a replacement for expert agricultural diagnosis. Therefore, the system displays confidence values and communicates uncertainty to help users interpret the results responsibly. By supporting early awareness of plant diseases and stress symptoms, SmartLeaf aims to contribute to sustainable agriculture, reduce unnecessary pesticide usage, and make AI-supported agricultural analysis more accessible.

This final report presents the completed architecture and design of the system, the implementation status, the tools and technologies used, the impact of the engineering solution, contemporary issues related to AI in agriculture, and the final test results based on the previously prepared test plan.

## 2. Introduction

Agriculture is one of the most important sectors for food security, economic stability, and environmental sustainability. Plant diseases, nutrient deficiencies, and late detection of stress symptoms may reduce crop yield and cause economic losses for farmers. In many agricultural environments, plant health inspection is still performed manually by farmers or agricultural experts. Although manual inspection is valuable, it can be time-consuming, inconsistent, and difficult to scale, especially when expert support is not easily available.

SmartLeaf was developed to address this problem by providing an automated, image-based plant health analysis system. The system uses leaf images as input and applies computer vision and machine learning techniques to identify visible disease symptoms, estimate the general health condition of the leaf, and provide possible nutrient deficiency indications. The goal is to support users with preliminary analysis results that can help them take earlier and more informed action.

The project was developed throughout CMPE 491 and CMPE 492 as a two-semester senior design project. In CMPE 491, the project idea, requirements, analysis model, and high-level design were

prepared. In CMPE 492, the design was refined into low-level components, implemented, tested, and prepared for final demonstration. The final system follows a modular web-based architecture consisting of a frontend layer, backend API layer, machine learning analysis layer, and reporting/output layer.

SmartLeaf does not aim to replace professional agronomists or laboratory-based plant diagnosis. Instead, it is designed as a decision-support platform. The system provides predictions and indicators with confidence values and encourages users to interpret the results as preliminary guidance. This responsible approach is important because AI-based systems may make incorrect predictions when image quality is low, when the plant type is outside the supported dataset, or when symptoms are visually ambiguous.

The final implementation demonstrates that modern web technologies and machine learning models can be combined to create an accessible agricultural support tool. By deploying the frontend on Vercel and the backend on Render, the system became available through a cloud-based environment without requiring users to install specialized software. This makes SmartLeaf practical for demonstration and suitable for future extension.

### 3. Project Overview

SmartLeaf is a web-based plant health analysis platform. The user interacts with the system through a simple web interface. The core workflow consists of the following steps:

1. The user opens the SmartLeaf website.
2. The user uploads a plant leaf image in a supported format.
3. The frontend sends the image to the backend API.
4. The backend validates the image format and size.
5. The image is processed by the machine learning analysis service.
6. The system generates a disease prediction, confidence score, health score, and possible nutrient deficiency indication.
7. The frontend displays the results in a structured and user-friendly format.
8. The user can review the analysis output and download or use the generated report if needed.

The system focuses on accessibility and simplicity. The interaction flow is intentionally kept short so that non-technical users can complete the analysis without needing prior experience with machine learning systems. The interface avoids overly technical terminology where possible and presents the output using clear labels, numerical scores, and explanatory messages.

The main features of SmartLeaf are:

- plant leaf image upload,
- file type and file size validation,
- image preprocessing,
- disease classification,
- confidence score generation,
- health score calculation,
- possible nutrient deficiency indication,

- result visualization,
- error handling for invalid inputs,
- PDF/report generation,
- cloud deployment through Vercel and Render.

The project was developed with future extensibility in mind. The modular architecture allows new disease classes, new plant species, improved machine learning models, user accounts, historical analysis, and mobile application support to be added in later versions.

## 4. Final Status of the Project

At the end of the CMPE 492 project period, SmartLeaf reached a functional implementation state. The final version includes the main features defined in the earlier project documents and demonstrates the complete workflow from image upload to result presentation.

The frontend application was implemented and deployed using Vercel. It provides the user-facing interface of the system, including the landing page, image upload area, result display components, and feedback messages. The frontend communicates with the backend service through API requests and presents the analysis results returned by the backend.

The backend service was implemented and deployed using Render. It is responsible for handling analysis requests, validating uploaded files, coordinating the image analysis workflow, and returning structured responses to the frontend. The backend also includes error handling mechanisms for invalid file types, oversized files, empty uploads, and analysis failures.

The machine learning related functionality was integrated into the backend workflow. The system analyzes uploaded leaf images and returns prediction-oriented outputs including disease label, confidence score, health score, and possible nutrient deficiency indication. The final implementation provides an end-to-end decision-support experience for users.

The reporting functionality was also implemented at a practical level. Users can obtain a structured summary of the analysis result, and the report generation feature supports the academic and practical goal of making the output shareable and understandable.

| Feature                        | Final Status       |
|--------------------------------|--------------------|
| Web-based frontend             | Implemented        |
| Frontend deployment            | Deployed on Vercel |
| Backend API                    | Implemented        |
| Backend deployment             | Deployed on Render |
| Image upload                   | Implemented        |
| File validation                | Implemented        |
| Disease classification         | Implemented        |
| Confidence score               | Implemented        |
| Health score                   | Implemented        |
| Nutrient deficiency indication | Implemented        |
| Result page                    | Implemented        |
| Error handling                 | Implemented        |

| Report/PDF generation | Implemented |  
| End-to-end workflow | Implemented and tested |

Although the main project goals were achieved, SmartLeaf still has limitations. The system is mainly designed for leaf image based analysis and its reliability depends on image quality, lighting conditions, plant type, and the representativeness of the dataset. The nutrient deficiency indication should be interpreted as a suggestive output rather than a definitive diagnosis. In future versions, the system can be improved with larger datasets, field testing, more disease classes, user history, mobile support, and expert feedback integration.

## **5. Final System Architecture and Design**

SmartLeaf follows a modular, layered, and service-oriented software architecture. The system is divided into separate components so that each layer has a clear responsibility and can be developed, tested, deployed, and maintained independently. This design supports maintainability, scalability, and future extensibility.

The final architecture consists of four main layers:

1. Presentation Layer
2. Backend API Layer
3. Machine Learning and Analysis Layer
4. Reporting and Output Layer

### **5.1 Presentation Layer**

The presentation layer is the frontend web application deployed on Vercel. It is responsible for all user-facing interactions. Users access SmartLeaf through a standard web browser without installing additional software.

The main responsibilities of the presentation layer are:

- displaying the landing page and project interface,
- allowing users to upload plant leaf images,
- performing basic client-side checks where applicable,
- sending image files to the backend API,
- displaying disease prediction, confidence score, health score, and nutrient deficiency indication,
- showing error messages and warnings,
- providing access to report output.

The frontend was designed to be simple and accessible. Since the target users may include farmers and non-technical users, the interface avoids unnecessary complexity and keeps the analysis workflow short.

### **5.2 Backend API Layer**

The backend API layer is deployed on Render and acts as the central coordinator of the system. It receives requests from the frontend, validates uploaded files, manages the analysis workflow, and returns structured JSON responses.

The main responsibilities of the backend layer are:

- receiving image upload requests,
- validating file type, file size, and empty file conditions,
- forwarding valid images to the analysis service,
- coordinating machine learning inference,
- calculating or organizing analysis outputs,
- handling errors in a controlled way,
- returning structured responses to the frontend,
- supporting health-check or system status behavior.

The backend follows RESTful API principles. This makes the system easier to integrate with the frontend and also allows future clients, such as a mobile application, to use the same backend service.

### **5.3 Machine Learning and Analysis Layer**

The machine learning and analysis layer is responsible for processing uploaded leaf images and generating analysis results. This layer performs image preprocessing, disease classification, confidence score generation, health score calculation, and possible nutrient deficiency indication.

The main responsibilities of this layer are:

- preparing image data for inference,
- loading and using the trained or selected model,
- classifying the leaf condition,
- generating confidence values,
- supporting health score calculation,
- producing possible nutrient deficiency indicators,
- returning raw analysis outputs to the backend service.

The machine learning layer was kept separate from user interface logic. This separation is important because the model can be improved, replaced, or extended in the future without redesigning the frontend.

### **5.4 Reporting and Output Layer**

The reporting and output layer is responsible for formatting analysis results in a user-friendly and shareable way. It structures the prediction result, confidence score, health score, and nutrient deficiency indication so that users can understand the output more easily.

The main responsibilities of this layer are:

- formatting analysis results,

- preparing report content,
- supporting PDF/report generation,
- ensuring consistent output structure,
- making analysis results easier to share or document.

This layer supports both academic project requirements and practical user needs. Farmers or researchers may want to keep records of plant health analysis results, and structured reporting helps address this need.

## 5.5 Final System Workflow

The final SmartLeaf workflow is as follows:

1. The user opens the SmartLeaf website.
2. The frontend loads from Vercel.
3. The user uploads a plant leaf image.
4. The frontend sends the image to the backend API deployed on Render.
5. The backend validates the file.
6. If the file is invalid, the backend returns an error response.
7. If the file is valid, the backend forwards it to the analysis workflow.
8. The analysis service performs preprocessing and inference.
9. The system generates disease classification, confidence score, health score, and nutrient deficiency indication.
10. The backend returns the structured result to the frontend.
11. The frontend displays the result page.
12. The user can review the result and generate or download the report.

This workflow provides a complete end-to-end experience and satisfies the core functional requirements defined earlier in the project.

## 5.6 Design Principles

**Separation of Concerns:** Frontend, backend, machine learning, and reporting responsibilities are separated.

**Modularity:** Each subsystem is designed as a modular part of the whole system.

**Scalability:** Using Vercel for frontend deployment and Render for backend deployment allows the system to be hosted in a cloud-based environment and supports future scaling.

**Maintainability:** The codebase follows a service-oriented structure.

**Responsible AI:** The system presents confidence values and does not claim to provide definitive diagnosis.

**User-Centered Design:** The workflow is kept simple so that users can complete an analysis with minimal effort.

## 6. Implementation Details

SmartLeaf was implemented as a web-based application with a separated frontend and backend structure. The implementation follows the architecture defined during the design phase and focuses on delivering the core project workflow: image upload, validation, analysis, result presentation, and reporting.

### 6.1 Frontend Implementation

The frontend was implemented as a modern web application and deployed on Vercel. It provides the visual interface of SmartLeaf and allows users to interact with the system through a browser.

The frontend includes the following main parts:

- landing or home page,
- upload interface,
- result display area,
- warning and error message components,
- report-related user interface elements.

The upload interface allows users to select an image file from their device. The selected image is sent to the backend API for analysis. After the backend returns the response, the frontend updates the result section and displays the disease label, confidence score, health score, and possible nutrient deficiency indication.

### 6.2 Backend Implementation

The backend was implemented as an API service and deployed on Render. It is responsible for handling requests from the frontend and managing the analysis workflow.

The backend includes the following core responsibilities:

- accepting image upload requests,
- checking whether the uploaded file is empty,
- validating supported image formats such as JPEG and PNG,
- enforcing maximum file size limits,
- calling the analysis service,
- handling machine learning related outputs,
- returning structured JSON responses,
- returning meaningful error messages when validation or analysis fails.

### 6.3 Image Validation

Image validation checks whether the uploaded file is empty, whether the content type is supported, and whether the file size is within the allowed limit. This validation contributes to system reliability and improves user feedback.

### 6.4 Analysis Service Implementation

The analysis service coordinates the main analysis workflow. It connects backend request handling logic with machine learning and result generation logic.

The analysis process includes receiving a valid image, preparing the image for analysis, sending the image to the model or prediction service, receiving prediction output, calculating or organizing additional outputs such as health score, and preparing the final response for the frontend.

## **6.5 Machine Learning Implementation**

The machine learning component generates prediction-oriented outputs from the uploaded leaf image. The implementation supports predicted disease or health label, confidence score, health score, and possible nutrient deficiency indication.

## **6.6 Health Score and Nutrient Deficiency Indication**

The health score provides a numerical representation of the estimated condition of the leaf on a 0–100 scale. The nutrient deficiency indication provides a possible suggestion based on visible symptoms. This output is presented as an indication rather than a final diagnosis.

## **6.7 Report Generation**

The reporting functionality allows the analysis result to be presented in a structured format. The report includes prediction label, confidence score, health score, and nutrient deficiency indication.

## **6.8 Deployment Implementation**

The final system was deployed using Vercel for frontend deployment and Render for backend deployment. This deployment structure separates the user interface from backend logic and makes the system accessible online.

# **7. Tools and Technologies Used**

During the development of SmartLeaf, several modern tools, frameworks, platforms, and libraries were used. These tools supported frontend development, backend implementation, machine learning integration, deployment, version control, and documentation.

## **7.1 Frontend Technologies**

- React / Next.js
- JavaScript / TypeScript
- HTML and CSS
- Responsive design principles
- Vercel

## **7.2 Backend Technologies**

- Python
- FastAPI
- Uvicorn
- Pydantic / Python type validation principles
- Render

### **7.3 Machine Learning and Image Processing Technologies**

- Python-based ML environment
- Deep learning / CNN-based classification approach
- Image preprocessing techniques
- Public agricultural datasets, especially PlantVillage-style datasets

### **7.4 Reporting Technologies**

SmartLeaf includes report generation functionality to present the analysis output in a structured format. The reporting component supports structured result formatting, user-readable output generation, PDF/report export functionality, and consistent presentation of analysis results.

### **7.5 Version Control and Collaboration Tools**

The team used Git and GitHub for source code management and collaboration. GitHub supported version tracking, team collaboration, code organization, repository-based delivery, and public access to source code when required by the course.

### **7.6 Documentation and Design Tools**

The documentation process included UML-based design thinking, requirement analysis, architecture documentation, testing documentation, report writing, and project website preparation.

## **8. Impact of Engineering Solutions**

SmartLeaf is not only a software project but also an engineering solution that addresses a real-world problem in agriculture. Plant health directly affects food production, farmer income, sustainability, and environmental protection.

### **8.1 Global Impact**

Food security is a global issue. Plant diseases and delayed diagnosis can reduce agricultural productivity and increase crop losses. In many regions, farmers may not have immediate access to agricultural experts or laboratory testing. SmartLeaf addresses this challenge by offering an accessible web-based plant health analysis tool.

The global impact of SmartLeaf is connected to precision agriculture. SmartLeaf contributes to this direction by showing how machine learning can be used to support plant disease awareness and early intervention.

### **8.2 Economic Impact**

SmartLeaf can reduce economic burden by providing a low-cost preliminary analysis tool. Since the system is web-based, users do not need specialized hardware.

The economic benefits of SmartLeaf include earlier awareness of plant health problems, reduced need for repeated manual inspection, better prioritization of expert consultation, potential reduction in unnecessary pesticide or fertilizer usage, and improved accessibility for small and independent farmers.

### **8.3 Environmental Impact**

SmartLeaf supports environmental sustainability by encouraging more informed decision-making. If users can receive preliminary indications about plant disease or stress, they may avoid unnecessary or random treatment attempts.

The environmental benefits include supporting early disease awareness, helping reduce unnecessary pesticide usage, encouraging targeted agricultural intervention, supporting sustainable crop management, and reducing crop waste caused by late detection.

### **8.4 Societal Impact**

SmartLeaf has a strong societal dimension because agriculture affects communities, food availability, employment, and local economies. The project aims to make AI-supported plant analysis more accessible through a simple web interface.

The societal benefits include making plant health analysis more accessible, supporting non-technical users, helping small farmers and agricultural students, increasing awareness of plant diseases and nutrient stress, and contributing to digital transformation in agriculture.

### **8.5 Ethical and Professional Impact**

SmartLeaf involves artificial intelligence, and AI systems must be designed responsibly. The ethical principles followed in SmartLeaf include transparency about uncertainty, avoiding definitive diagnosis claims, protecting user-uploaded image data, minimizing unnecessary storage, designing for accessibility, and acknowledging model limitations.

## **9. Contemporary Issues Related to the Project**

### **9.1 Artificial Intelligence in Agriculture**

Artificial intelligence is increasingly used in agriculture for disease detection, yield prediction, irrigation planning, weed detection, soil monitoring, and crop management. SmartLeaf is part of this larger trend.

### **9.2 Dataset Bias and Model Generalization**

A model trained on a limited dataset may perform well on similar images but poorly on real-world images from different regions, lighting conditions, or plant varieties. SmartLeaf addresses this issue by presenting confidence scores and treating results as suggestive.

### **9.3 Responsible AI and Human Oversight**

SmartLeaf follows a human-in-the-loop perspective. The system provides analysis, but the final decision belongs to the user or an agricultural expert.

### **9.4 Sustainability and Climate Change**

Climate change increases the importance of sustainable agriculture. SmartLeaf contributes to sustainability by supporting early detection and more informed decisions.

### **9.5 Digital Divide**

Although SmartLeaf is designed to be accessible, digital divide remains an important issue. Future versions can include mobile application support, offline or low-connectivity mode, multilingual interface, simpler user guidance, and integration with agricultural extension services.

### **9.6 Data Privacy**

SmartLeaf processes user-uploaded plant images. The system minimizes data storage and focuses on temporary processing. If user accounts or historical tracking are added in the future, stronger privacy policies and secure storage practices will be required.

## **10. Use of Library and Internet Resources**

The development of SmartLeaf required the use of library resources, Internet resources, academic references, documentation, datasets, and software engineering materials.

### **10.1 Background Research**

The team used background research to understand plant disease detection, image classification, and AI-supported agriculture. PlantVillage-style resources were important for understanding plant disease image datasets.

### **10.2 Software Engineering Resources**

Software engineering resources guided project specification, requirement analysis, high-level design, low-level design, test planning, final implementation, and final testing.

### **10.3 Technical Documentation**

The team used documentation for React / Next.js, FastAPI, Python, Vercel, Render, machine learning and image processing libraries, GitHub, and Git.

## 10.4 Engineering Principles

The project used engineering principles such as modularity, abstraction, separation of concerns, validation, testing, reliability, and maintainability.

## 10.5 References Used in Previous Reports

The project benefited from professional ethics resources, software engineering textbooks, REST architecture resources, and deep learning materials.

## 11. Test Results

This section presents the final test results of SmartLeaf based on the test cases defined in the Test Plan Report. The tests cover image upload, input validation, preprocessing, disease classification, health score generation, nutrient deficiency indication, result display, report generation, error handling, performance, consistency, data privacy, and the complete end-to-end workflow.

### 11.1 Test Results Table

| Test ID | Test Name                      | Brief Description                                                        | Result                 | Notes                                                                                                                                        |
|---------|--------------------------------|--------------------------------------------------------------------------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| TC-001  | Valid Image Upload             | A valid JPEG/PNG plant leaf image is uploaded through the web interface. | Passed                 | The system accepted valid image files and forwarded them to the backend for analysis.                                                        |
| TC-002  | Unsupported File Format        | A non-image file such as .txt or .pdf is uploaded.                       | Passed                 | The system rejected unsupported file formats and displayed an appropriate error message.                                                     |
| TC-003  | Oversized File Upload          | An image larger than the allowed file size limit is uploaded.            | Passed                 | The backend validation rejected oversized files and returned a size-related error response.                                                  |
| TC-004  | Image Preprocessing            | A valid image is processed before inference.                             | Passed                 | Uploaded images were prepared for analysis through the preprocessing workflow.                                                               |
| TC-005  | Disease Classification         | A diseased leaf image is uploaded and analyzed.                          | Passed                 | The system returned a disease/condition label with a confidence score.                                                                       |
| TC-006  | Health Score Validation        | A valid image is uploaded and the generated health score is checked.     | Passed                 | The health score was generated within the expected 0–100 range.                                                                              |
| TC-007  | Nutrient Deficiency Indication | An image with visible stress or deficiency symptoms is uploaded.         | Passed with limitation | The system produced a possible nutrient deficiency indication. This output is suggestive and not a definitive diagnosis.                     |
| TC-008  | Result Page Display            | The analysis result is displayed on the frontend.                        | Passed                 | Disease label, confidence score, health score, and nutrient indication were displayed clearly.                                               |
| TC-009  | PDF / Report Export            | The user generates or downloads a structured report after analysis.      | Passed                 | The report functionality produced a structured summary of the analysis result.                                                               |
| TC-010  | Invalid Image Handling         | A corrupted or unreadable image is uploaded.                             | Passed                 | The system handled invalid image input without crashing and returned an error message.                                                       |
| TC-011  | Low Confidence Warning         | An unclear or low-quality image is uploaded.                             | Passed with limitation | The system communicated uncertainty through confidence information. More advanced low-quality image detection can be improved in the future. |

| TC-012 | End-to-End Workflow | Complete workflow: upload image, analyze, view result, and generate report. | Passed | The complete SmartLeaf workflow was successfully executed. |  
| TC-013 | Performance Test | A standard image is uploaded and response time is observed. | Passed | Under normal conditions, analysis was completed within an acceptable response time. Cloud cold starts may occasionally increase response time. |  
| TC-014 | Repeated Input Consistency | The same image is uploaded multiple times. | Passed | The system produced similar results for repeated identical inputs. |  
| TC-015 | Data Privacy Check | Uploaded image handling is checked after analysis. | Passed | The system follows temporary processing principles and does not require permanent storage of uploaded images for normal analysis. |

## 11.2 Test Result Summary

| Result Category | Number of Test Cases | Percentage |  
| Passed | 13 | 86.7% |  
| Passed with limitation | 2 | 13.3% |  
| Failed | 0 | 0% |  
| Total | 15 | 100% |

The majority of the test cases passed successfully. No critical failing test case remained at the end of the final testing stage. Two test cases were marked as “Passed with limitation” because they involve outputs that should be interpreted carefully: nutrient deficiency indication and low-confidence/unclear image handling.

## 11.3 Functional Testing Assessment

Functional testing showed that SmartLeaf satisfies the main functional requirements defined in earlier project documents. The system successfully supports image upload, validation, analysis, result display, and report generation.

## 11.4 Non-Functional Testing Assessment

Non-functional testing focused on performance, reliability, usability, privacy, and consistency. The performance test showed that the system responds within an acceptable time under normal usage conditions. Since the backend is deployed on Render, cloud cold starts may occasionally increase response time.

## 11.5 Error Handling and Negative Testing Assessment

Unsupported file format testing showed that files such as text documents or PDFs are not accepted as image input. Oversized file testing showed that file size limits are enforced. Corrupted image testing showed that the system can return an error response instead of crashing.

## 11.6 Bugs Observed During Testing

During testing and implementation, several minor issues were observed and addressed:

- Some invalid files initially caused backend-side processing errors before validation was improved.

- Large image files increased response time and required file size validation.
- Frontend result display needed consistent formatting for confidence and health score values.
- Cloud deployment required configuration adjustments for frontend-backend API communication.
- Backend cold start behavior on Render occasionally increased the first request response time.

No unresolved critical bug remained at the end of the final testing stage.

## **11.7 Limitations Identified Through Testing**

- The system performs best with clear leaf images.
- Poor lighting, blurred images, or complex backgrounds may reduce prediction reliability.
- Nutrient deficiency indication is suggestive and should not be interpreted as a definitive diagnosis.
- The system is mainly focused on leaf image analysis and may not perform reliably on full plant or branch images.
- Real-world agricultural field validation is limited within the scope of the course project.
- The current deployment is suitable for demonstration and moderate use but not optimized for large-scale production traffic.

## **12. Test Assessment and Future Enhancements**

The final testing process indicates that SmartLeaf achieved its core implementation goals. All planned test cases were executed, and no failed test case remained in the final result table.

### **12.1 Overall Test Assessment**

The test results show that SmartLeaf is functionally complete for the defined project scope. The system satisfies the primary requirements: image upload, invalid input handling, backend validation, analysis result generation, result display, report generation, end-to-end workflow, and cloud deployment.

### **12.2 Discussion of Passed with Limitation Test Cases**

TC-007 Nutrient Deficiency Indication: The system provides possible nutrient deficiency indication, but this result should not be treated as a final agricultural diagnosis.

TC-011 Low Confidence Warning: The system communicates uncertainty through confidence score information. However, a more advanced image quality detection module could improve the handling of unclear, blurred, or irrelevant images.

### **12.3 Potential Enhancements**

Future versions can include a larger and more diverse dataset, mobile application support, advanced image quality detection, severity estimation, historical tracking, expert feedback integration, multilingual support, and production-level scalability.

### **12.4 Final Testing Conclusion**

The testing process confirmed that SmartLeaf is a functional and complete senior project implementation. The system meets the main functional requirements and demonstrates a realistic engineering solution for AI-assisted plant health analysis.

## **13. Project Website and GitHub URLs**

### **13.1 Project Website**

Project Website:

<https://smart-leaf-beta.vercel.app/>

### **13.2 GitHub Repositories**

Combined Project Repository:

<https://github.com/morethanyouth1nk/smartleaf>

Frontend Directory:

<https://github.com/morethanyouth1nk/smartleaf/tree/main/frontend>

Machine Learning Directory:

<https://github.com/morethanyouth1nk/smartleaf/tree/main/ml>

Backend Files:

<https://github.com/morethanyouth1nk/smartleaf>

## **14. User Manual and Installation Instructions**

### **14.1 User Manual**

1. Open the SmartLeaf website.
2. Navigate to the image upload section.
3. Select a plant leaf image from the device.
4. Make sure the image is in a supported format such as JPEG or PNG.
5. Submit the image for analysis.
6. Wait for the system to process the image.
7. View the analysis result on the result page.
8. Review the disease label, confidence score, health score, and possible nutrient deficiency indication.
9. If needed, generate or download the structured report.

## 14.2 Input Guidelines for Users

Users should use a clear image of a plant leaf, avoid blurry or low-resolution images, take the image under sufficient lighting, keep the leaf visible and centered, avoid complex backgrounds if possible, upload only supported image formats, and avoid files larger than the allowed limit.

## 14.3 Interpretation of Results

SmartLeaf provides decision-support outputs. The results should not be interpreted as a definitive agricultural diagnosis. For serious crop problems, users should consult an agricultural expert or agronomist.

## 14.4 Local Installation Requirements

To run SmartLeaf locally, Node.js and npm, Python, required Python packages, Git, and internet connection are required.

## 14.5 Frontend Installation

```
```bash
cd frontend
npm install
npm run dev
```
```

The frontend is usually available at:

```
```bash
http://localhost:3000
```
```

## 14.6 Backend Installation

```
```bash
cd backend
pip install -r requirements.txt
python3 -m uvicorn main:app --reload
```
```

The backend is usually available at:

```
```bash
http://localhost:8000
```
```

## 14.7 Environment Configuration

Example environment variable:

```
```bash
NEXT_PUBLIC_API_URL=http://localhost:8000
```
```

For deployment, this value should be changed to the deployed backend API URL.

## 14.8 Deployment Instructions

The frontend can be deployed on Vercel by connecting the GitHub repository and configuring environment variables. The backend can be deployed on Render by creating a new web service, connecting the repository, installing dependencies, and setting the start command.

A typical backend start command is:

```
```bash
python3 -m uvicorn main:app --host 0.0.0.0 --port $PORT
```
```

## 14.9 Troubleshooting

| Problem                            | Possible Solution                                                 |
|------------------------------------|-------------------------------------------------------------------|
| Frontend cannot connect to backend | Check API URL environment variable.                               |
| Backend does not start             | Check Python version and installed dependencies.                  |
| Image upload fails                 | Check file format and file size.                                  |
| Analysis takes too long            | Wait for backend cold start or try again with a smaller image.    |
| Result is uncertain                | Upload a clearer image with better lighting.                      |
| PDF/report does not generate       | Check backend report generation route and frontend button action. |

## 15. Conclusion

SmartLeaf was developed as an AI-based plant health analysis system that allows users to upload leaf images and receive preliminary analysis results. The system combines frontend development, backend API implementation, machine learning based image analysis, result visualization, report generation, and cloud deployment.

The final implementation achieved the main goals defined in the earlier project phases. The system provides image upload, file validation, disease classification, confidence score, health score, possible nutrient deficiency indication, result display, and report generation. The frontend was deployed on Vercel and the backend was deployed on Render, creating a functional cloud-based demonstration environment.

From an engineering perspective, SmartLeaf demonstrates the use of modular architecture, RESTful communication, input validation, service-oriented backend design, machine learning integration, and structured testing. The system was designed with maintainability and future extensibility in mind.

From a societal perspective, SmartLeaf addresses an important agricultural need. Plant health problems can affect food production, farmer income, environmental sustainability, and community welfare. By providing an accessible decision-support tool, SmartLeaf contributes to the digital transformation of agriculture and supports early awareness of plant disease and stress symptoms.

The project also emphasizes responsible AI usage. SmartLeaf does not present its results as definitive diagnosis. Instead, it communicates uncertainty through confidence scores and presents nutrient deficiency output as a possible indication. This approach helps reduce the risk of misinterpretation and supports human oversight.

Final testing showed that the system satisfies the core functional requirements and provides a complete end-to-end workflow. Although limitations remain, especially in real-world field generalization, image quality handling, and advanced nutrient deficiency diagnosis, the final system successfully demonstrates the feasibility and value of AI-supported plant health analysis.

Overall, SmartLeaf is a functional, socially relevant, and technically meaningful senior design project. It provides a strong foundation for future development and shows how computer engineering solutions can contribute to sustainable agriculture and public benefit.

## **16. References**

- [1] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv:1511.08060, 2015.
- [2] PlantVillage Dataset. Available: <https://plantvillage.psu.edu/>
- [3] ACM Code of Ethics and Professional Conduct. Available: <https://www.acm.org/code-of-ethics>
- [4] IEEE Code of Ethics. Available: <https://www.ieee.org/about/corporate/governance/p7-8.html>